

Evaluation-Efficient Design of a Tow-Thomas Bandpass Filter Using TabPFN-based Bayesian Optimisation

Abstract—Active analog filters are key building blocks in analog signal processing, and the Tow-Thomas biquad is a versatile topology that can realize several filter responses. This study compares four optimisation methods for an active Tow-Thomas bandpass filter design: three Bayesian optimisation (BO) variants, each using a different surrogate model (a Gaussian process, TabPFN with bootstrap uncertainty, and TabPFN with native quantile uncertainty), and simulated annealing as a model-free reference. All four methods share the same initialization and evaluation budget, and the three BO variants run inside an identical optimisation loop, so the surrogate model is the only experimental variable. Bayesian optimisation with TabPFN native quantile uncertainty achieved the best final performance, with a median cost of 0.0274 against 0.0360 for simulated annealing, while using only about one-fifth of the surrogate computation of the bootstrap variant. Our results indicate that surrogate-based optimisation performs best under a tight evaluation budget, and that Bayesian optimisation with TabPFN native quantile uncertainty is the preferable choice for the considered filter test circuit.

Index Terms—Active filter design, Bayesian optimisation, circuit optimisation, EDA, Gaussian processes, TabPFN, simulated annealing.

I. INTRODUCTION

ANALOG ACTIVE FILTERS are core building blocks in communication, instrumentation, and signal-processing systems, where they pass a desired band and reject unwanted components. Designing one means choosing component values, resistors and capacitors, so that the circuit meets a target frequency response while remaining stable and manufacturable with standard parts [1], [2]. Since filter performance sets the accuracy of later signal-processing stages, precise component selection is critical, making filter design a recurring and practically significant optimisation problem.

Choosing the right component values for a filter is not that straightforward. Textbook design formulae assume ideal components and ideal op-amps, so they ignore parasitics, finite gain-bandwidth, and loading effects that noticeably shift the response of a real circuit [3]. To capture these effects, the circuit must be evaluated with a SPICE-level simulator. Components are also restricted to standard discrete series (E12 resistors, preferred-value capacitors), so the design space is large and combinatorial rather than continuous. For example, the test filter circuit considered in this study is an 8-component bandpass filter, which alone admits about 4.3×10^{11} configurations. Some configurations make the simulation fail to converge, creating discontinuities in the objective. Simulator-

in-the-loop evaluation, discrete variables, and a discontinuous landscape together rule out gradient-based methods and make brute-force search infeasible.

Classical approaches use metaheuristics such as simulated annealing (SA), genetic algorithms, and differential evolution [2], [4], [5]; these are simple and assumption-free but extract no model from past evaluations, so they typically need many evaluations to converge. Bayesian optimisation (BO) targets the opposite trade-off: it fits a cheap probabilistic surrogate from the evaluations seen so far and uses an acquisition function to pick the next point, balancing exploitation of promising regions against exploration of uncertain ones [6]–[9]. The open question is not whether BO works here, but under what budget it beats a well-configured metaheuristic, and which surrogate to use.

In this work, we compare, under a fair protocol, a Gaussian process (GP) surrogate against tabular prior-data fitted network (TabPFN), a transformer that performs in-context learning and returns a predictive distribution without per-problem training [10], [11]. All methods share the same initial sample space and a 500-evaluation budget, and the three BO variants share one loop in which only the surrogate differs; SA is warm-started from the same pool. Across six runs we find a two-regime picture: the BO methods lead when the budget is small (at 200 evaluations, GP-BO and TabPFN-bootstrap beat SA on five of six runs), SA catches up once the full budget is spent, and TabPFN native quantile uncertainty ends with the best median while running about five times faster than its bootstrap counterpart. This budget-dependent view and the quantile-versus-bootstrap comparison are the main contributions of this paper.

II. PROBLEM FORMULATION

A. Tow-Thomas Biquad Active Filter

The Tow-Thomas biquad is a widely used second-order active filter that realizes a bandpass response using three op-amps arranged as two integrators and an inverter in a feedback loop [12]. It is a standard textbook topology, valued for its low sensitivity to component variation and for allowing the center frequency and quality factor to be tuned almost independently, which makes it a common choice in practical audio and instrumentation filters [2]. For bandpass operation, the transfer

function is:

$$H(s) = \frac{-\frac{s}{\omega_0 Q}}{s^2 + \frac{s}{\omega_0 Q} + \omega_0^2}$$

$$\omega_0 = \frac{1}{\sqrt{R_5 R_6 C_1 C_2}}, \quad Q = \sqrt{\frac{R_6 C_2}{R_5 C_1}}$$

The circuit is simulated in PySpice with ideal VCVS op-amps (gain 10^5) and high-impedance bias resistors for numerical stability. AC analysis runs from 50 Hz to 20 kHz at 300 points per decade.

B. Design Parameters and Objective Function

We optimise 8 discrete components: 6 resistors from the E12 series (100 Ω - 100 k Ω , 37 values) and 2 capacitors from a custom series (1 - 100 nF, 13 values), giving search space $\mathcal{X} = \mathcal{R}^6 \times \mathcal{C}^2$ with $|\mathcal{X}| \approx 4.3 \times 10^{11}$.

Sampling range justification. These ranges follow the filter’s operating-point physics: the center frequency obeys $f_c \approx 1/(2\pi RC)$, so a 1 kHz target needs $RC \approx 1.6 \times 10^{-4}$ s (e.g. $R = 10$ k Ω , $C = 15$ nF). The E12 resistor and capacitor ranges bracket this operating point by about two orders of magnitude in each direction, covering the realizable RC products while excluding physically irrelevant regions.

Objective. The target is a 1 kHz bandpass filter with rejection at 200 Hz and 5 kHz. Given frequency response $|H(f)|$ from SPICE:

$$J(\mathbf{x}) = \left(1 - \frac{|H(f_{\text{target}})|}{|H|_{\text{max}}}\right)^2 + 2.5 \left[\left(\frac{|H(f_{\text{low}})|}{|H|_{\text{max}}}\right)^2 + \left(\frac{|H(f_{\text{high}})|}{|H|_{\text{max}}}\right)^2 \right]$$

where $|H|_{\text{max}} = \max_f |H(f)|$. The first term rewards unity gain at target; the second penalizes leakage at rejection frequencies (weighted $2.5\times$). Simulation failures receive penalty $J = 10^6$. The optimisation objective is $\mathbf{x}^* = \arg \min_{\mathbf{x} \in \mathcal{X}} J(\mathbf{x})$.

Together, the tight evaluation budget, discrete variables, simulation failures that create discontinuities, and a multimodal landscape make this a challenging optimisation problem.

III. OPTIMISATION ALGORITHMS

All methods need starting information before they can search. We generate one shared set of initial points with Latin Hypercube Sampling (LHS), a stratified technique that divides each parameter range into equal-probability intervals and draws one sample per interval, giving space-filling coverage with few points [13]. Samples are drawn in $[0, 1]^8$, mapped to component space via log-scale interpolation, and snapped to the nearest discrete value. The 30 LHS points are SPICE-evaluated once and handed identically to every method: the three BO variants use them as their initial dataset, and SA is warm-started from the best of the same pool. These 30 evaluations count against every method’s budget of 500

SPICE evaluations. In industrial practice each evaluation is expensive: transient analyses, Monte-Carlo corners, and post-layout extraction cost seconds to minutes. Our small test circuit takes only about 1.4 ms per AC analysis, which makes a controlled multi-run study feasible; we therefore treat the number of simulator calls, not wall-clock time, as the unit of optimisation cost.

A. A Shared Bayesian Optimisation Loop

Bayesian optimisation maintains a probabilistic surrogate $\mathcal{M}$ of the objective and, at each iteration, uses the surrogate’s predictive mean $\mu(\mathbf{x})$ and standard deviation $\sigma(\mathbf{x})$ to decide where to evaluate next. The next points are chosen by Expected Improvement (EI) for minimization [6]:

$$\alpha_{\text{EI}}(\mathbf{x}) = (J_{\min} - \mu(\mathbf{x}) - \xi) \Phi(Z) + \sigma(\mathbf{x}) \phi(Z)$$

$$Z = \frac{J_{\min} - \mu(\mathbf{x}) - \xi}{\sigma(\mathbf{x})}$$

where $J_{\min}$ is the best observed cost, Φ and ϕ are the standard normal CDF and PDF, and $\xi \geq 0$ controls exploration. EI is zero when $\sigma = 0$.

To make the comparison meaningful, the BO loop is implemented exactly once (Algorithm 1) and the three BO methods differ *only* in the surrogate that supplies (μ, σ) . Candidates, the context cap, the acquisition function, and top- K verification are all shared, so the surrogate choice is the sole experimental variable.

Algorithm 1 Shared BO loop (surrogate $\mathcal{M}$ is the plug-in)

Require: objective J , search space $\mathcal{X}$, surrogate $\mathcal{M}$, shared LHS data $\mathcal{D}_0$ ($n_0 = 30$), budget $N = 500$

- 1: **while** SPICE evaluations used $< N$ **do**
- 2: $\mathcal{D}_{\text{ctx}} \leftarrow \text{CapContext}(\mathcal{D}, 150)$
- 3: Fit $\mathcal{M}$ on $\mathcal{D}_{\text{ctx}}$ in log-cost space
- 4: $\mathbf{X}_{\text{cand}} \leftarrow \text{MixedCandidates}(\mathcal{X}, 500)$
- 5: $\mu, \sigma \leftarrow \mathcal{M}.\text{predict}(\mathbf{X}_{\text{cand}})$
- 6: $\alpha \leftarrow \alpha_{\text{EI}}(\mu, \sigma, \log(1 + J_{\min}))$
- 7: $S \leftarrow$ top-3 candidates by α
- 8: With probability 0.1: add one uniformly random candidate to S
- 9: **for** $\mathbf{x} \in S$ **do**
- 10: $y \leftarrow J(\mathbf{x})$ via SPICE $\triangleright$ counts against N
- 11: add $(\mathbf{x}, y)$ to $\mathcal{D}$ unless the simulation failed
- 12: **end for**
- 13: **end while**
- 14: **return** $\arg \min_{\mathbf{x} \in \mathcal{D}} J(\mathbf{x})$

In Algorithm 1, $\mathcal{D}$ is the dataset of all evaluated designs and $J_{\min}$ the best cost seen so far. The pseudocode calls two subroutines, detailed below: $\text{CapContext}(\mathcal{D}, 150)$ subsamples $\mathcal{D}$ to at most 150 points before the surrogate is fitted, and $\text{MixedCandidates}(\mathcal{X}, 500)$ proposes 500 trial designs to score. The acquisition α_{EI} is the Expected Improvement defined above, and “top-3” keeps the three candidates with the highest α_{EI} for SPICE verification.

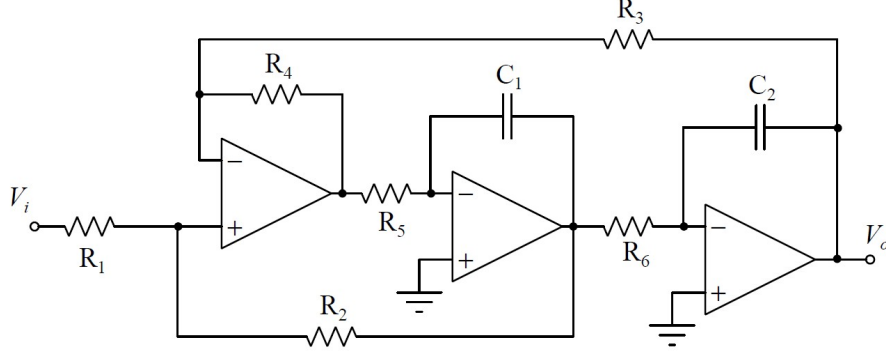


Fig. 1. Second-order Tow-Thomas biquad filter topology used in this study (adapted from [2]).

Candidate generation (MixedCandidates). Each iteration draws 500 mixed candidates: 50% local perturbations around the current best points, 30% stratified random samples, and 20% hard-negative samples biased toward extreme regions. This pool injects diversity even when the surrogate becomes confident.

Context subsampling (CapContext). When $|\mathcal{D}| > 150$, the surrogate context keeps the 80% lowest-cost points plus 20% random ones:

$$\mathcal{D}_{\text{ctx}} = \text{Top}_{0.8n_{\text{max}}}(\mathcal{D}, \text{by cost}) \cup \text{Random}_{0.2n_{\text{max}}}(\mathcal{D} \setminus \text{Top})$$

Verification and log-cost space. The surrogate predicts $\log(1 + J)$ to stabilize variance; EI is computed in this space and the top- $K = 3$ candidates are verified with real SPICE simulations. Failed simulations (penalty 10^6) count against the budget but are excluded from the surrogate context, so the model is not distorted by the penalty plateau.

Exploration injection. With probability 0.1 per iteration, one extra verification is spent on a uniformly random candidate. This guards against EI collapse when an overconfident surrogate predicts near-zero σ everywhere; it costs about 3% of the budget and is applied identically to all three BO methods.

B. Surrogate Models

1) *Gaussian Process*: The GP surrogate defines a distribution over functions, $f(\mathbf{x}) \sim \mathcal{GP}(\mu(\mathbf{x}), k(\mathbf{x}, \mathbf{x}'))$, whose posterior mean and variance at new points have closed forms [14]. We use a Matérn-5/2 kernel with per-dimension length scales (ARD) plus a white-noise term, and refit the GP from scratch on the capped context at every iteration. The GP is fully online: there is no offline pre-training and no warm start beyond the shared LHS data. Fitting on at most 150 points is cheap ($O(n^3)$ with $n \leq 150$), so refitting every iteration costs little and keeps the model current.

2) *TabPFN*: TabPFN is a transformer-based meta-learner trained on synthetic regression tasks that performs in-context learning: given a context $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ and test points, it outputs a predictive distribution in a single forward pass, with no gradient updates on the user’s data [10]. Its advantages

over a GP in this role are that it needs no kernel choice or hyperparameter tuning, requires no per-problem training, and provides distributional outputs natively. We use TabPFN v2, which supports up to 10,000 context samples and 500 features [11]; the context cap of 150 in our loop is our own design choice for speed, not a model limit. We evaluate two ways of extracting the uncertainty that EI needs.

TabPFN-Bootstrap refits TabPFN on $B = 5$ bootstrap resamples of the context (each resample draws $0.8n$ points with replacement) and takes the spread of the resulting mean predictions as the uncertainty (Algorithm 2): the ensemble mean is $\mu_{\text{ens}} = \frac{1}{B} \sum_b \hat{\mu}_b$ and the standard deviation is $\sigma_{\text{ens}} = \text{std}_b(\hat{\mu}_b)$, i.e. the disagreement between the B resample fits. This captures how sensitive the prediction is to the composition of the context, but it costs B separate fit-and-predict passes per iteration.

Algorithm 2 TabPFN-Bootstrap Prediction

Require: context $(\mathbf{X}, \mathbf{y})$ of size n , test points $\mathbf{X}^*$, $B = 5$

- 1: **for** $b = 1$ to B **do**
 - 2: $I_b \leftarrow \text{draw } \lceil 0.8n \rceil$ indices with replacement
 - 3: fit TabPFN on $(\mathbf{X}[I_b], \mathbf{y}[I_b])$
 - 4: $\hat{\mu}_b \leftarrow \text{TabPFN.predict}(\mathbf{X}^*)$
 - 5: **end for**
 - 6: $\mu_{\text{ens}} \leftarrow \frac{1}{B} \sum_{b=1}^B \hat{\mu}_b$
 - 7: $\sigma_{\text{ens}} \leftarrow \text{std}(\hat{\mu}_1, \dots, \hat{\mu}_B)$ ▷ elementwise over b
 - 8: **return** $\mu_{\text{ens}}, \sigma_{\text{ens}}$
-

TabPFN-Quantile instead reads the uncertainty directly off TabPFN’s native predictive distribution, requesting the quantiles $q_{0.1}, q_{0.5}, q_{0.9}$ in a single forward pass (Algorithm 3):

$$\begin{aligned} \mu(\mathbf{x}) &= q_{0.5}(\mathbf{x}) \\ \sigma(\mathbf{x}) &\approx \frac{q_{0.9}(\mathbf{x}) - q_{0.1}(\mathbf{x})}{2.56} \end{aligned} \tag{1}$$

where 2.56 converts the 10th-90th percentile range to a standard deviation under normality. This removes the factor- B overhead entirely. More principled uncertainty extraction for prior-data fitted networks is an active research topic [15], but the simple quantile recipe is considered here.

C. Simulated Annealing as a Reference

SA is included as a model-free reference, to show what the surrogate machinery actually buys. SA accepts worse solutions with Metropolis probability $P = \exp(-\Delta E/T)$, allowing escape from local minima at high temperature [4], [5]. We use geometric cooling $T_{t+1} = \alpha T_t$ with $\alpha = 0.95$. The neighborhood has two levels: 85% of moves perturb a single component by 1-2 positions in its discrete value list, and 15% are full random restarts that resample every component.

Two details keep SA on the same footing as the BO methods. First, SA starts from the best point of the shared LHS pool rather than a single random configuration, and the 30 initial evaluations count against its budget exactly as they do for BO. Second, the initial temperature is scaled to the warm start, $T_0 = 0.5 \times J(\mathbf{x}_0)$: a fixed cold-start temperature such as $T_0 = 2$ would accept a +0.4-worse move with probability around 82% and immediately random-walk away from the good starting point, which would be an unfair handicap rather than a property of the algorithm. The rest of the procedure is the standard warm-started Metropolis loop as reported in [4], [5].

IV. IMPLEMENTATION AND EXPERIMENTAL SETUP

All experiments use PySpice 1.5 (ngspice 34) for circuit simulation with ideal VCVS op-amps (gain = 10^5), Scikit-learn 1.8 for the GP surrogate, and TabPFN 2.2.1 for the transformer surrogates. LHS sampling uses SciPy 1.16 and the code runs on Python 3.13. Hardware: Intel i5-13450HX, 16 GB RAM; all surrogates run on CPU.

A. Fairness Protocol

To ensure a fair comparison, the following measures were adopted:

- 1) **Shared initialization.** One pure-LHS set of 30 points is drawn once, evaluated once, and given identically to all methods. No method receives extra domain-biased or offline samples.
- 2) **Equal budget in evaluations.** Every method runs to the same cap of 500 SPICE evaluations, initial points included, with no early stopping. The budget is counted in evaluations, not iterations, because one BO iteration costs 3 simulations (occasionally 4, with the exploration injection) while one SA iteration costs 1; counting iterations would hand the BO methods a hidden $3\times$ budget. 500 evaluations correspond to 500 SA iterations or roughly 150 BO iterations.

Algorithm 3 TabPFN-Quantile Prediction

Require: fitted TabPFN, test points $\mathbf{X}^*$

- 1: $\mathbf{Q} \leftarrow \text{TabPFN.predict_quantiles}(\mathbf{X}^*, [0.1, 0.5, 0.9])$
 - 2: $\boldsymbol{\mu} \leftarrow \mathbf{Q}[:, 1]$ $\triangleright$ median = $q_{0.5}$
 - 3: $\boldsymbol{\sigma} \leftarrow (\mathbf{Q}[:, 2] - \mathbf{Q}[:, 0]) / 2.56$ $\triangleright q_{0.9} - q_{0.1}$ to std
 - 4: **return** $\boldsymbol{\mu}, \boldsymbol{\sigma}$
-

- 3) **Identical BO loop.** The loop of Algorithm 1 is implemented once; only the (μ, σ) predictor differs between GP-BO, TabPFN-Bootstrap, and TabPFN-Quantile.
- 4) **Repetition.** The whole comparison is repeated over six independent runs (1-6). Within each run, the same random seed is used by all the algorithms to control both the shared LHS draw and every stochastic decision inside the optimisers.

B. Feature Preprocessing

Component values span orders of magnitude, so we apply log-scale normalization:

$$x_i^{\text{norm}} = \frac{\log_{10}(x_i) - \log_{10}(x_{\min})}{\log_{10}(x_{\max}) - \log_{10}(x_{\min})}$$

mapping all features to $[0, 1]$. Costs are transformed as $y_{\log} = \log(1 + y)$ to stabilize variance. The same preprocessing feeds every surrogate.

C. Hyperparameter Settings

Table I lists the hyperparameter configurations for all the optimisation algorithms. The common settings are listed in row 1 while row 2 gives the BO loop settings. Rows (3-5) refer to the per-surrogate settings and SA parameters are given in row 6.

TABLE I
HYPERPARAMETER CONFIGURATIONS.

Scope	Parameter	Value
All methods	Init. samples	30 (shared LHS)
	SPICE budget	500 evals (init incl.)
	Early stopping	none
	Runs	6 (1-6)
BO (shared)	Acquisition	EI ($\xi = 0.01$), log-cost
	Candidates/iter	500 (mixed sampling)
	SPICE sims/iter	3 (top- K)
	Exploration inject	$p = 0.1$ (+1 random sim)
	Context cap	150 (80% best + 20% rand.)
GP-BO	Kernel	Matérn-5/2 ARD + WN
	Bootstraps	$B = 5, 0.8 n$ resamples
TabPFN-Q	Quantiles	$[0.1, 0.5, 0.9]$, 1 pass
SA	Start	best of shared LHS pool
	Init. temp.	$T_0 = 0.5 \times \text{start cost}$
	Cooling rate	$\alpha = 0.95$
	Moves	85% local, 15% restart
	SPICE sims/iter	1

Performance metrics. With six runs per method, we report the median and interquartile range (IQR) of the final cost, the best final cost over all runs (labelled as ‘Best (run)’), the number of runs on which each method finishes with the best or lowest cost (labelled as ‘Wins’), and the median wall-clock time (labelled as ‘Time’). To expose budget effects, we also tabulate the median best-so-far cost at evaluation budgets from 50 up to the full 500.

V. RESULTS AND COMPARISON

A. Convergence Behavior

Fig. 2 shows the median convergence trajectories with IQR bands over the six runs, plotted against cumulative SPICE evaluations so that all methods are on the same cost axis.

Table II gives the median best-so-far cost at fixed budget cutoffs, which is the clearest summary of the study.

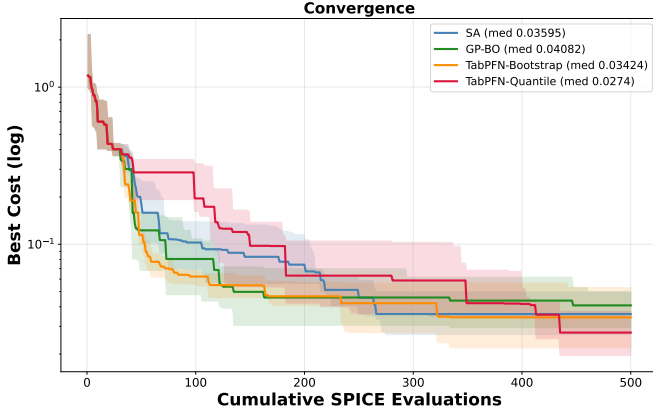


Fig. 2. Median convergence over six runs (shaded: interquartile range) versus cumulative SPICE evaluations.

TABLE II
MEDIAN BEST-SO-FAR COST OVER SIX RUNS AT FIXED EVALUATION BUDGET CUTOFFS (BOLD MARKS A MEDIAN VALUE LOWER THAN OBTAINED BY SA AT THE SAME BUDGET).

Evals	SA	GP-BO	TabPFN-BS	TabPFN-Q
50	0.180	0.123	0.115	0.287
100	0.103	0.081	0.062	0.196
150	0.083	0.050	0.055	0.098
200	0.074	0.046	0.047	0.063
300	0.036	0.046	0.042	0.059
500	0.036	0.041	0.034	0.027

Two regimes are visible. **Under tight budgets** (up to about 200 evaluations) the surrogate-based methods lead: GP-BO and TabPFN-Bootstrap have a lower median cost than SA at every checkpoint from 50 evaluations onward, and by 200 evaluations all three BO variants beat SA's median. At the per-run level, GP-BO and TabPFN-Bootstrap each reach a lower cost than SA on five of the six runs at the 200-evaluation mark. **Given the full budget**, SA catches up: between 200 and 300 evaluations its median drops sharply from 0.074 to 0.036 and then flatlines. The mechanism is easy to see in the algorithm. By this stage the temperature has decayed to essentially zero ($T_0 \approx 0.2$ scaled by $0.95^{200} \approx 3 \times 10^{-5}$), so uphill acceptance is gone and SA degenerates into a greedy local search plus its 15% random restarts, which act as a keep-if-better random search over the whole space. Given hundreds of evaluations, that simple strategy is hard to beat on this landscape. The surrogate advantage is concentrated below about 250 evaluations.

TabPFN-Quantile is the interesting case: it is the *slowest* starter (median 0.196 at 100 evaluations, worse than SA) but improves steadily through the whole budget and finishes with the best median cost, 0.0274. Its early sluggishness reflects the narrow native-quantile spread on small contexts: the spread is tight, EI turns mostly exploitative, and progress waits until the context becomes informative enough for the median predictions to point somewhere useful.

B. Solution Quality

Table III summarizes the final costs. The medians of all four methods land within a factor of 1.5 of each other (0.027-0.041), which is itself a finding: with a fair protocol and a 500-evaluation budget, no method dominates. TabPFN-Quantile has the best median and the tightest IQR, TabPFN-Bootstrap wins the most individual runs (3 of 6), and SA is fully competitive with both BO-with-GP and BO-with-bootstrap at this budget.

TABLE III
FINAL COST OVER SIX RUNS AT THE FULL 500-EVALUATION BUDGET.

Method	Median	IQR	Best (run)	Wins	Time (s)
SA	0.0360	[0.027, 0.050]	0.0191 (5)	1	0.7
GP-BO	0.0408	[0.029, 0.050]	0.0172 (1)	0	212
TabPFN-BS	0.0342	[0.022, 0.053]	0.0051 (1)	3	3039
TabPFN-Q	0.0274	[0.020, 0.037]	0.0032 (5)	2	616

Table IV gives the per-run final costs behind these summaries. TabPFN-Quantile produced both the single best design of the whole study (0.0032 on run 5) and its own worst case (0.112 on run 1, a run that stalled early and never recovered), while SA never finished worse than 0.065 on any run. Run-to-run variation is of the same order as the differences between methods, which is why we report medians and wins rather than single-run numbers.

TABLE IV
PER-RUN FINAL COSTS (BOLD: LOWEST PER RUN). NO METHOD WINS ON MORE THAN HALF THE RUNS.

Run	SA	GP-BO	TabPFN-BS	TabPFN-Q
1	0.0379	0.0172	0.0051	0.1122
2	0.0340	0.0641	0.0214	0.0227
3	0.0646	0.0331	0.0234	0.0321
4	0.0241	0.0502	0.0451	0.0391
5	0.0191	0.0282	0.0670	0.0032
6	0.0536	0.0485	0.0556	0.0185

Fig. 3 shows the frequency responses of each method's best-of-runs design (best run per method). All four produce a recognizable 1 kHz bandpass; the TabPFN-Quantile design has the deepest stopband floor (about -35 dB on the low side versus -22 to -23 dB for GP-BO and SA), which is exactly where the $2.5\times$ -weighted rejection terms of the cost function reward improvement. Table V lists the component values of the best-of-runs design obtained by each method: SA and TabPFN-Quantile from run 5, GP-BO and TabPFN-Bootstrap from run 1.

TABLE V
COMPONENT VALUES CORRESPONDING TO BEST-OF-RUNS DESIGN OBTAINED BY EACH METHOD.

Comp.	GP-BO	Tab-BS	Tab-Q	SA
R_1 (k Ω)	15.00	3.90	0.33	2.20
R_2 (k Ω)	27.00	82.00	47.00	5.60
R_3 (k Ω)	1.20	56.00	68.00	5.60
R_4 (k Ω)	0.15	0.10	2.20	0.82
R_5 (k Ω)	82.00	5.60	82.00	22.00
R_6 (k Ω)	0.39	0.12	0.10	0.33
C_1 (nF)	10.00	68.00	100.00	33.00
C_2 (nF)	10.00	1.00	1.00	15.00

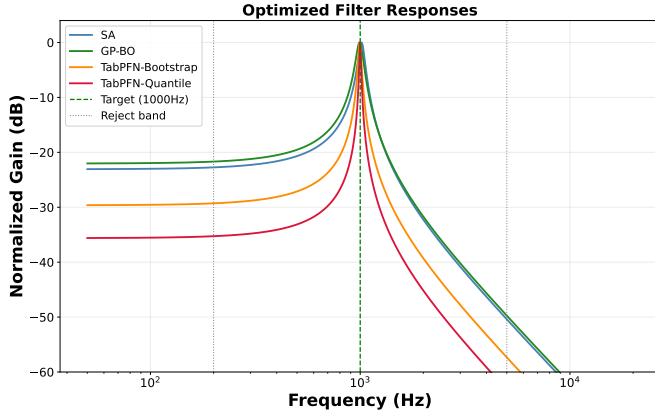


Fig. 3. Normalized frequency responses of best-of-runs design obtained by each method (best run per method shown). TabPFN-Quantile (red) achieves the deepest rejection at 200 Hz and 5 kHz.

C. Computational Efficiency

All methods spend the same 500 simulations, so wall-clock differences are pure surrogate overhead. SA completes a run in about 0.7 s (raw simulation cost ≈ 1.4 ms/eval), GP-BO adds about 212 s, TabPFN-Quantile about 616 s, and TabPFN-Bootstrap about 3039 s. The bootstrap-to-quantile ratio is $4.9\times$, matching the factor $B = 5$: bootstrap runs five fit-and-predict passes where quantile runs one. Since TabPFN-Quantile also achieves the better median (0.0274 versus 0.0342), it is the better choice here. For genuinely expensive simulations (seconds to minutes each) these overheads become negligible and only sample efficiency (Table II) matters.

D. Computational Complexity

The measured timings follow from the per-iteration structure. Let $m = 500$ be the candidate count, $n \leq 150$ the context size, $B = 5$ the number of bootstraps, and $C_{\text{spice}} \approx 1.4$ ms the cost of one simulation. Per iteration, GP-BO refits the GP in $O(n^3)$ and predicts in $O(mn)$; each TabPFN forward pass is dominated by self-attention over the context, $O(mn^2)$, with the quantile variant using one pass and bootstrap using B ; SA performs one simulation and $O(1)$ bookkeeping. Over T iterations,

$$\mathcal{T}_{\text{GP-BO}} \approx O(T(n^3 + mn)), \quad \mathcal{T}_Q \approx O(Tmn^2), \quad \mathcal{T}_{\text{BS}} \approx B \mathcal{T}_Q.$$

With n capped at 150, the measured per-iteration times, about 1.4 s (GP-BO), 4 s (quantile), and 20 s (bootstrap) on CPU, reproduce the factor B , while SA's total time equals the raw simulation cost $O(N C_{\text{spice}})$ of the whole budget.

VI. CONCLUSION

We compared four optimisers for SPICE-in-the-loop design of a Tow-Thomas biquad bandpass filter under a controlled protocol: shared LHS initialization, an equal budget of 500 SPICE evaluations, six runs, and a single BO loop in which the surrogate is the only difference between the Bayesian methods. The outcome is a two-regime picture. Under tight

budgets, the surrogates performed better: by 200 evaluations all three BO variants beat simulated annealing's median cost, and GP-BO and TabPFN-Bootstrap do so on five of six runs. Over the full budget, SA catches up through its restart mechanism, and the final medians of all methods fall within a factor of 1.5, with TabPFN-Quantile best at 0.0274 versus a value of 0.0360 obtained by simulated annealing. The clearest single recommendation concerns the surrogate itself: TabPFN's native quantile uncertainty delivered a better median final cost than bootstrap ensembling at about one fifth of the surrogate compute. Our results suggest that a surrogate-based optimisation method has the best performance under tight budgets and BO with TabPFN-Quantile as the most suitable optimisation algorithm for the considered test filter circuit. These conclusions rest on one circuit topology and six runs (the 5-of-6 pattern reaches only a one-sided sign-test $p \approx 0.11$); extending the protocol to more topologies and to multi-objective, Pareto-optimal optimisation is the natural continuation of this work.

REFERENCES

- [1] R. Schaumann and M. E. Van Valkenburg, *Design of Analog Filters*, 2nd ed. Oxford University Press, 2001.
- [2] B. Durmuş, "Optimal components selection for active filter design with average differential evolution algorithm," *AEU International Journal of Electronics and Communications*, vol. 94, pp. 293-302, 2018.
- [3] A. S. Sedra and K. C. Smith, *Microelectronic Circuits*, 7th ed. Oxford University Press, 2015.
- [4] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by Simulated Annealing," *Science*, vol. 220, no. 4598, pp. 671-680, 1983.
- [5] S. Geman and D. Geman, "Stochastic Relaxation, Gibbs Distributions, and the Bayesian Restoration of Images," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-6, no. 6, pp. 721-741, 1984.
- [6] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas, "Taking the human out of the loop: A review of Bayesian optimization," *Proceedings of the IEEE*, vol. 104, no. 1, pp. 148-175, 2016.
- [7] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian Optimization of Machine Learning Algorithms," *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 2951-2959, 2012.
- [8] P. I. Frazier, "A Tutorial on Bayesian Optimization," *arXiv preprint arXiv:1807.02811*, 2018.
- [9] A. I. J. Forrester, A. Söbester, and A. J. Keane, *Engineering Design via Surrogate Modelling: A Practical Guide*, Wiley, 2008.
- [10] N. Hollmann, S. Müller, K. Eggensperger, and F. Hutter, "TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second," *International Conference on Learning Representations (ICLR)*, 2023. <https://doi.org/10.48550/arXiv.2207.01848>
- [11] N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirmer, and F. Hutter, "Accurate predictions on small data with a tabular foundation model," *Nature*, vol. 637, pp. 319-326, 2025.
- [12] J. Tow, "Design formulas for active RC filters using operational-amplifier biquad," *Electronics Letters*, vol. 5, no. 15, pp. 339-341, Jul. 1969.
- [13] M. D. McKay, R. J. Beckman, and W. J. Conover, "A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code," *Technometrics*, vol. 21, no. 2, pp. 239-245, 1979.
- [14] C. K. I. Williams and C. E. Rasmussen, *Gaussian Processes for Machine Learning*, MIT Press, 2006.
- [15] N. Hollmann, M. Wistuba, and F. Hutter, "Uncertainty Quantification for Prior-Data Fitted Networks using Martingale Posteriors," *arXiv preprint arXiv:2505.11325*, 2025. <https://doi.org/10.48550/arXiv.2505.11325>